

Java Inheritance (Abstract Classes & Interfaces) Extra Practice Practical Solutions

Note

These are merely proposed solutions. You may provide other and even better ones!

Exercise 1: Implementing an Abstract Class

```
abstract class Payment {
    protected double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    public abstract void processPayment();
}

class CreditCardPayment extends Payment {

    private String cardNumber;

    public CreditCardPayment(double amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing credit card payment of $" +
            amount + " from card: " + cardNumber);
    }
}

class PayPalPayment extends Payment {

    private String email;

    public PayPalPayment(double amount, String email) {
        super(amount);
        this.email = email;
    }
}
```

```
@Override
public void processPayment() {
    System.out.println("Processing PayPal payment of $" +
        amount + " to email: " + email);
}
}

public class PaymentSystem {

    public static void main(String[] args) {

        Payment creditPayment = new CreditCardPayment(100.0,
            "1234-5678-9876-5432");
        Payment paypalPayment = new PayPalPayment(50.0, "user@example.com");

        creditPayment.processPayment();
        paypalPayment.processPayment();
    }
}
```

Exercise 2: Implementing an Interface

```
interface Refundable {
    void refund();
}

class CreditCardPayment extends Payment implements Refundable {

    private String cardNumber;

    public CreditCardPayment(double amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing credit card payment of $" +
            amount + " from card: " + cardNumber);
    }

    @Override
    public void refund() {
        System.out.println("Refunding $" + amount +
            " to credit card: " + cardNumber);
    }
}

public class RefundablePaymentSystem {

    public static void main(String[] args) {
        Payment payment = new CreditCardPayment(100.0, "1234-5678-9876-5432");
        payment.processPayment();

        if (payment instanceof Refundable) {
            ((Refundable) payment).refund();
        }
    }
}
```

Exercise 3: Combining Abstract Classes and Interfaces

```
interface DigitalWallet {
    void walletInfo();
}

class PayPalPayment extends Payment implements DigitalWallet {

    private String email;

    public PayPalPayment(double amount, String email) {
        super(amount);
        this.email = email;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing PayPal payment of $" +
            amount + " to email: " + email);
    }

    @Override
    public void walletInfo() {
        System.out.println("PayPal account: " + email);
    }
}
```

Exercise 4: Multiple Inheritance using Interfaces

```
interface CurrencyConverter {
    void convertCurrency(double rate);
}

class PayPalPayment implements CurrencyConverter {

    @Override
    public void convertCurrency(double rate) {
        System.out.println("Converted amount: " + (100 * rate));
    }
}
```

Exercise 5: Payment Processing System

```
abstract class Payment {
    protected double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    public abstract void processPayment();
    public void transactionReceipt() {
        System.out.println("Transaction amount: "+amount+" at DATE/TIME");
    }
}
```

```
interface MobilePayment {
    public boolean mobileTransaction();
}
```

```
class GooglePayPayment extends Payment implements MobilePayment {

    private String google_account;

    public GooglePayPayment(double amount, String google_account) {
        super(amount);
        this.google_account = google_account;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing GooglePay payment of $" +
            amount + " to account: " + google_account);
    }

    @Override
    public boolean mobileTransaction() {
        System.out.println("This is a mobile transaction.");
        return true;
    }
}
```